

Inside coding agents

How the runtime turns model output into action

Codex

Gemini CLI

Claude Code

Four layers of an agent runtime

- | | | |
|-----------|------------------------------|---|
| 01 | Interface | Input, streamed output and approvals |
| 02 | Session controller | History, model requests and turn completion |
| 03 | Tool runtime | Validation, scheduling and result handling |
| 04 | Execution environment | Files, processes, MCP and OS isolation |

One task across several model requests

01

INSPECT

Find the failure

Read files. Run a test.
Return observations
with call IDs.

02

CHANGE

Apply an edit

Validate arguments.
Check policy. Execute
in the allowed environment.

03

VERIFY

Check the outcome

Run the test again.
Report success, error
or cancellation.

Model requests and tool execution

Model service

Receives context and tool definitions.
Returns text or proposed tool calls.

Agent runtime

Owns session state and continuation.
Interprets calls and applies policy.

Execution environment

Accesses files and runs processes.
Returns tool observations to the runtime.

User interface

Displays progress and requests approval.
Accepts input and interruption.

What all three systems share

An iterative control loop

Model output can request tools.
Tool results inform the next request.

An execution boundary

The runtime mediates tool access.
The model proposes actions.

Context management

History and instructions consume space.
Long tasks need context reduction.

An extension surface

Tools, instructions and hooks broaden work.
Available interfaces vary by product.

The shared tool-call lifecycle

- | | | |
|-----------|------------------|---|
| 01 | Interpret | Resolve the tool name and arguments. |
| 02 | Authorize | Apply the configured policy and confirmation. |
| 03 | Execute | Run the action in its allowed environment. |
| 04 | Observe | Return success, failure or cancellation. |

Approval and sandboxing

Authorization decides whether to proceed. Isolation limits access.

Boundary	Question	Example
Validation	Are arguments valid?	Required path or command
Approval	May this action run?	Ask, allow or deny
Sandbox	What can it access?	Filesystem and network limits
Result	What happened?	Exit, error or cancellation

Shared problem: finite context

A safe history boundary

Gemini chooses a split point that keeps tool calls and results coherent.

Recent tool output has its own budget.

```
findCompressSplitPoint(...)
```

A summary loses detail

Keep constraints and verification evidence in durable artifacts.

Conversation history cannot roll back a filesystem change.

The same runtime supports broader work

Tools change the domain

Shell, files and MCP support code, documents and services.

Delegation adds a context

Give the child a scoped task.
Evaluate its returned evidence.

Skills change the instructions

Task guidance enters context.
The host still executes tools.

External writes need authority

Repository edits are inspectable.
Remote side effects need explicit control.

The evidence boundary

Two open implementations. One documented commercial runtime.

System	Public evidence	What we can inspect
Codex	Apache-2.0 source	Rust core, CLI and app-server
Gemini CLI	Apache-2.0 source	TypeScript CLI and core
Claude Code	Docs, examples, plugins	Public hooks, not the private core

Different ways to organize the runtime

Comparable responsibilities do not imply identical internal objects.

Concern	Gemini CLI	Codex	Claude Code
Conversation	GeminiClient and GeminiChat	Session and TurnContext	Documented sessions
Model exchange	Turn.run event adapter	Sampling step and StepContext	Documented loop Private core
Tool execution	Scheduler and ToolInvocation	Router, registry and tool runtimes	Public hooks and tool contracts

The model stream becomes typed events

SIMPLIFIED PSEUDOCODE

```
01 for await (const event of stream) {  
02     if (signal.aborted) return;  
03     yield contentEvent(event);  
04     for (const call of calls(event)) {  
05         yield toolRequestEvent(call);  
06     }  
07 }
```

**Text and tool
requests
share a stream**

Turn records pending calls.

Retry, cancellation and
completion remain separate
events.

The controller decides when a turn ends

DIRECT SOURCE EXCERPT

```
01 let needs_follow_up =  
02     model_needs_follow_up  
03     || has_pending_input;
```

**Tool work or new
input
can keep a turn alive**

run_turn also checks context
limits and compaction.

Stream completion and task
completion differ.

Concurrency is a concrete design difference

These are the verified mechanisms in the inspected snapshots.

System	Mechanism	Practical implication
Gemini CLI	Adjacent eligible calls form a batch	Edits and update_topic stay serial. wait_for_previous controls ordering.
Codex	Shared read guards or an exclusive write guard	Tool metadata selects admission. Cancellation can race with completion.
Claude Code	Private scheduling implementation	The public repository cannot establish an equivalent algorithm.

Shared or exclusive tool admission

DIRECT SOURCE EXCERPT

```
01 let _guard = if supports_parallel {  
02     Either::Left(lock.read().await)  
03 } else {  
04     Either::Right(lock.write().await)  
05 };
```

**One gate,
two access modes**

Read guards admit parallel tools.

A write guard admits one exclusive tool.

Completed results survive cancellation.

Dispatch and execution policy

Which tool handles the call?

ToolRouter resolves the name.

ToolRegistry dispatches the invocation.

ToolCallRuntime controls admission.

May the action run?

ToolOrchestrator resolves approval.

It selects the execution sandbox.

Retries follow the applicable policy.

A hook turns policy into executable code

SHORTENED PUBLIC EXAMPLE

```
01 input_data = json.load(sys.stdin)
02 command = input_data.get(
03     "tool_input", {}).get("command", "")
04 issues = _validate_command(command)
05 if issues:
06     sys.exit(2)
```

Exit 2 blocks the call

PreToolUse matches Bash.

The hook reads JSON on stdin
and sends feedback on stderr.

This example checks command
style.

Different runtimes, the same test cases

Malformed or denied calls

Unknown tool. Invalid arguments.
A coherent result for every call.

History integrity

Match results to call IDs.
Keep valid compaction boundaries.

Cancellation races

Cancel before admission or after completion.
Preserve completed results.

Retries and side effects

Repeat a failed exchange.
Check for duplicate writes.

What the implementations reveal

A source-code comparison, not a model-quality ranking.

Concern	Gemini CLI	Codex	Claude Code
Core	TS classes and events	Rust session and step state	Documented runtime
Concurrency	Adjacent eligible batches	Shared / exclusive admission	Private implementation
Deepest evidence	Scheduler and tool API	Turn and execution runtime	Public hook contract

DISCUSSION

**The tool finished.
The user pressed stop.**

**Which result should the next
model request receive?**

Where to put a breakpoint

Exact commit links and line anchors are in the speaker notes.

Project	Start here	Then inspect
Gemini CLI	core/client.ts	core/turn.ts
Gemini CLI	scheduler/scheduler.ts	tools/tools.ts
Codex	session/turn.rs	tools/parallel.rs
Codex	tools/router.rs	tools/orchestrator.rs
Claude Code	examples/hooks/	bash_command_validator_example.py

Snapshot and reading conventions

Pinned revisions

Codex ce2c2759ebee
Gemini ed2ac40df67a
Claude 347b38e4a733

Retrieved 9 September 2026

Evidence labels

Simplified code is pseudocode.

Direct excerpts retain identifiers.

Design implications are analysis.
Private implementations remain
unknown.